

AUDIO_DSP

Mutable Instruments 算法移植与 STM32H7 完整使用手册

范围	27 个 Eurorack 工程审计, 17 个工程 C API, 1 个 FSK 支持 API
目标	STM32H7 Cortex-M7, 裸机或 RTOS, SAI/I2S DMA
工具链	GNU Arm 13.3.1 + Keil ArmClang 6.24
验证	双工具链 0 error / 0 warning; 全部模块同镜像链接通过
上游快照	08460a69a7e1f7a81c5a2abcc7189c9a6b7208d4

本手册面向第一次接触实时音频 DSP 的读者。它不仅列出函数，还解释音频数据如何从 ADC/SAI DMA 进入算法、状态如何跨回调保存、控制参数何时更新、固定采样率如何处理，以及为什么大缓冲必须放在正确的 H7 存储区。

目录

目录	2
1. 结论与边界	4
1.1 官方来源核验	4
1.2 什么叫“移植正确”	4
2. 从零理解 H7 实时音频	4
2.1 一块音频是怎样流动的	4
2.2 初始化、控制和音频三条时间线	5
2.3 推荐的 48 帧 DMA 骨架	5
3. STM32H7 内存、DMA 与 Cache	5
3.1 为什么不能把大对象放局部变量	5
3.2 DCache 的正确边界	5
3.3 外部工作区对齐	6
4. 采样率与块长	6
4.1 不能用丢样代替采样率转换	6
5. 工具链与工程集成	6
5.1 GNU Arm CMake	6
5.2 Keil ArmClang	6
5.3 线程与中断安全	7
6. 模块逐项使用	7
6.1 Plaits 24 引擎宏振荡器	7
6.2 Braids 宏振荡器	8
6.3 Elements 物理建模音源	9
6.4 Rings 共振器与波导弦	10
6.5 Clouds 颗粒与频谱处理器	11
6.6 Warps 交叉调制、环调与声码器	12
6.7 Peaks 包络、LFO 与鼓合成	13
6.8 Tides 一代斜率发生器	14
6.9 Tides 2 四输出斜率发生器	15
6.10 Stages 多段包络与序列器	16
6.11 Streams 动态与控制处理	17
6.12 Frames 关键帧与四路 LFO	18
6.13 Marbles 相关随机门与随机电压	19
6.14 Grids 鼓点地图与欧几里得节奏	20
6.15 Branches 伯努利门路由	21
6.16 Edges 风格芯片音振荡器	22
6.17 Yarns 自适应纯律与 MIDI 时钟	23
6.18 FSK 解调与 CRC 数据包	24
7. 27 个上游工程总表	25
8. 对电赛信号题的实际作用	26
9. 常见故障定位	26
10. 上板验收清单	27
11. 许可证与发布	27

1. 结论与边界

- 原桌面 eurorack-master 解压目录的 1,109 个根仓库文件与官方 Git 对象逐一哈希一致，缺失 0、内容不一致 0；多出的 4 个文件仅为 Clouds 的 IDE 配置。
- 原下载的 5 个 Git 子模块目录为空，因此不能称为完整源码。已从官方仓库重新浅克隆并递归取得固定提交；原解压目录未被覆盖。
- 可复用算法以原上游 C++ 核心保存，外层提供 HAL 无关的 extern C API。这样避免重新手译数万行 DSP 引入数值差异，同时 C 主工程仍能直接调用。
- 17 个有数字固件的 Eurorack 项目提供 C API；10 个纯模拟/无固件项目在清单中明确标记。另公开音频引导程序的 FSK/CRC 算法。
- 当前证据证明源码可由两套 H7 工具链无警告编译，并可由 GNU 链接器把所有模块放进同一镜像；尚未替代具体开发板上的听音、频谱、时限和内存压力测试。

1.1 官方来源核验

子模块	固定提交
avrlib	276b2887e4110ca913294fcb313163dfb28a448
avrlibx	868d6e74eb8555f9575b5149202256156e5c03fa
avr_audio_bootloader	0de2be5928afdf7fb3e5ada60544ed6c15ab808f
stm_audio_bootloader	1ec7d6374e2d305cb710170eb80de9b66e795792
stmllib	e3bd7c9cc00e4364166f9905c0509b6ffd0535ec

官方根仓库固定为 commit **08460a69a7e1f7a81c5a2abcc7189c9a6b7208d4**。算法库的 module_manifest.json 保存同一证据，后续升级上游时应先改变清单提交，再重新做差异审计，不能把不同年代文件混在一起。

1.2 什么叫“移植正确”

1. 算法源保持上游实现和资源表，不擅自把浮点、定点或查表逻辑改写成“看起来等价”的 C。
2. 公开结构不暴露 C++ 类型，C11 文件能包含全部头文件；上下文由调用者静态持有，不用 malloc/new。
3. 所有原生块长约束由包装层处理或明确返回错误；事件与持续 gate 不混淆。
4. 工具链实际检查上下文尺寸；GNU 和 ArmClang 类布局不同，因此公开空间按两者较大值留余量。
5. 最终还要在目标板测 CPU 周期、栈余量、DMA cache、一致采样率和音频结果。前四项通过不等于第五项自动通过。

2. 从零理解 H7 实时音频

2.1 一块音频是怎样流动的

编解码器以固定采样率连续产生样点。SAI/I2S 外设由 DMA 把样点写入双缓冲；当一半缓冲填满时，中断只告诉 CPU “这一半可以处理”。DSP 函数不是自己去读取 ADC，也不会等待声音到来，它只处理你交给它的一段数组。

Codec ADC	SAI RX DMA	格式转换	DSP 核心	取幅/格式转换	SAI TX DMA
连续模拟输入	ping/pong 半缓冲	int16 或 float	状态跨块保留	避免溢出	连续模拟输出

例如采样率 48 kHz、DMA 半缓冲 48 帧，回调间隔就是 $48/48000 = 1\text{ ms}$ 。你必须在下一次半缓冲到来前处理完；平均够快但偶尔超过 1 ms 仍会爆音。

2.2 初始化、控制和音频三条时间线

- 初始化只做一次：分配静态状态/延迟线，调用 init，选择初始模式。不要在音频中断里反复 init。
- 控制通常 1 kHz 左右：扫描按键、电位器、MIDI，做去抖与平滑，把结果写入一个参数快照。
- 音频按 DMA 回调运行：复制一次参数快照，处理整个半块。不要在每个样点读取易变的 UI 全局变量。

2.3 推荐的 48 帧 DMA 骨架

```
static aud_euro_plaits_t synth;
static aud_euro_plaits_engine_memory_t synth_mem;
static aud_euro_plaits_patch_t patch;
static aud_euro_plaits_modulations_t mod;

void AudioApp_Init(void) {
    aud_euro_plaits_default_patch(&patch);
    aud_euro_plaits_default_modulations(&mod);
    mod.trigger_patched = 1;
    aud_euro_plaits_init(&synth, synth_mem.storage, sizeof(synth_mem.storage));
}

void Audio_ProcessHalf(int16_t *tx_interleaved, size_t stereo_frames) {
    aud_euro_plaits_patch_t p = patch;          /* one stable snapshot */
    aud_euro_plaits_modulations_t m = mod;
    aud_euro_plaits_process(
        &synth, &p, &m,
        (aud_euro_stereo_i16_t *)tx_interleaved,
        stereo_frames);
}
```

48 帧同时是 Plaits 的 24 帧整数倍和 Warps 的 3 帧整数倍，适合作为通用半缓冲。若使用 32 kHz Clouds/Elements，则 32 帧也对应 1 ms，但不能与 48 kHz 数据直接混用。

3. STM32H7 内存、DMA 与 Cache

3.1 为什么不能把大对象放局部变量

Rings 和 Elements 的上下文超过 100 KiB，若写在函数内部，它们会进入 MSP/PSP 或 RTOS 任务栈，通常立即导致 HardFault。所有 aud_euro_*_t、混响、颗粒和引擎工作区都应声明为 static/global，或由一个长期存在的音频对象持有。

存储区	CPU 特性	DMA	推荐用途
DTCM	M7 紧耦合、低延迟	通常不可由外设 DMA 直接访问	小型算法状态、热数据；不要放 SAI DMA 缓冲
AXI SRAM	容量较大，经过 cache/总线	依芯片与总线矩阵而定	Clouds/Rings/Elements 大状态与延迟线
D2 SRAM	适合外设总线访问	常用于 SAI/SPI DMA	RX/TX ping-pong 缓冲
外部 SDRAM	容量大、延迟较高	依 FMC/MDMA 配置	颗粒长缓冲；先实测带宽和 cache

不同 STM32H7 型号的容量和 DMA 可达性不同，必须以具体芯片参考手册和 CubeMX 生成的链接脚本为准。示例中的 .aud_fast 与 .aud_large 只是命名段，必须在 scatter/linker 文件里映射。

3.2 DCache 的正确边界

1. DMA 写 RX 缓冲后、CPU 读取前，对 cacheable 区域做 invalidate；地址和长度按 32 字节 cache line 对齐。
2. CPU 写 TX 缓冲后、DMA 读取前，做 clean。
3. 更简单的做法是通过 MPU 把 DMA 音频缓冲区设为 non-cacheable；算法内部状态仍可 cacheable。
4. 不要每块清理整个 DCache，也不要对仅 CPU 使用的 Clouds/Rings 延迟线做无意义 cache 维护。

3.3 外部工作区对齐

Clouds 和 Plaits 会把字节工作区解释成 float/uint16 等类型。公开的 aud_euro_clouds_*_buffer_t 与 aud_euro_plaits_engine_memory_t 自带 8 字节对齐；init 也会拒绝未对齐裸指针。DMA 缓冲还应额外按 32 字节对齐。

4. 采样率与块长

模块	原生/配置速率	块长规则	48 kHz 系统建议
Braids	96 kHz	内部 <=24	2 倍上/下采样或把 codec 设 96 kHz
Clouds	32 kHz	内部 <=32	2/3 与 3/2 有理数 SRC
Elements	32 kHz	内部 <=16	2/3 与 3/2 SRC
Stages	31.25 kHz	包装层 <=32	独立 32 us 控制定时器并插值
Peaks/Plaits/Rings/Tides	48 kHz	24 或包装层自动分块	直接使用
Warp	可配置	frames 必须为 3 倍数	48 帧 DMA 半块
Edges/Marbles/Tides2	可配置	见各 API	init 传实际速率

4.1 不能用丢样代替采样率转换

从 48 kHz 到 32 kHz 的比率是 2/3。正确流水线是先插值 2 得到 96 kHz，经低通 FIR 后每 3 点取 1 点；返回 48 kHz 时先插值 3，再低通并每 2 点取 1 点。直接按 3 点取 2 点会把 16 kHz 以上频谱折叠进可听带。

若只用一个固定采样率模块，最稳妥是直接配到它的原生速率。若要在同一链中混合多个速率，使用经验证的 polyphase SRC 或 CMSIS-DSP FIR interpolation/decimation，并把滤波器群延迟计入系统延迟。

5. 工具链与工程集成

5.1 GNU Arm CMake

```
cmake -S eurorack_algorithms -B build/eurorack_h7 -G Ninja \
  -DCMAKE_TOOLCHAIN_FILE=eurorack_algorithms/cmake/arm-none-eabi-h7.cmake \
  -DCMAKE_BUILD_TYPE=Release
cmake --build build/eurorack_h7
```

5.2 Keil ArmClang

1. 选择 Arm Compiler 6，目标 CPU Cortex-M7，FPU fpv5-d16，hard float。
2. 可用 cmake/armclang-h7.cmake 生成 libaud_eurorack_h7.a，或按 CMakeLists.txt 的源文件清单加入 Keil Group。不要把 vendor 下所有整机 drivers/main 一股脑编译。
3. 包含目录依次添加 eurorack_algorithms/include、port/include、vendor。
4. 定义 GCC_ARMCM7、ARM_MATH_CM7、__FPU_PRESENT=1、AUD_EURO_DISABLE_PERSISTENCE=1；ArmClang 还需 M_PI=3.14159265358979323846。
5. 关闭 exceptions、RTTI、threadsafe statics；打开 -O2、fast-math、function/data sections。
6. C 主文件只 include aud_eurorack.h；链接器仍必须包含 C++ 运行库和数学库。当前包装层自身不使用堆。

本机已用 Keil ArmClang 6.24 对 154 个编译/归档步骤实测 0 error、0 warning；GNU Arm 13.3.1 完成 156 步并把所有模块链接为一个 412,424 B text 的冒烟镜像。

5.3 线程与中断安全

- 一个上下文只能由一个音频执行线程写。UI 任务通过参数快照通信，不要同时调用 process。
- Grids 上游是全局单例，全固件只能有一个实例。Plaits 用户数据注册使用进程内桥接变量，多个 Plaits 实例应串行处理。
- 不要在音频 ISR 调用 printf、文件系统、Flash 擦写、malloc/free 或阻塞锁。

6. 模块逐项使用

以下顺序按“音源、效果、控制/节奏、通信”混合编排。每个模块都给出它解决什么问题、数据格式、内存、采样率、最短调用路径和最常见错误。完整可编译 C 示例位于 `eurorack_algorithms/examples/`。

6.1 Plaits 24 引擎宏振荡器

用途	覆盖虚拟模拟、波折叠、两/六算子 FM、粒子、和弦、弦乐、物理建模、鼓、人声和用户波表，是最适合直接做 H7 合成器的模块。
输入输出	无音频输入；双声道 int16 主/辅输出；trigger 是持续门电平。
内存	上下文 9,024 B + 对齐引擎工作区 16,384 B；可选外部用户数据页 4,096 B。
速率	固定 48 kHz；内部始终按 24 帧运行，包装层 FIFO 接受任意 DMA 长度。

最短正确调用流程

1. 静态声明 context 与 `aud_euro_plaits_engine_memory_t`，先取 default patch/modulations 再 init。
2. patch.engine 选择 0..23；note 是 MIDI 音符，harmonics/timbre/morph 为 0..1。
3. 若 trigger_patched=1，trigger 必须在按键期间持续为 1，释放时变 0。
4. 在 48 kHz SAI DMA 回调调用 process，输出结构与交错 int16 立体声布局一致。
5. 用户波表/DX7 数据用 set_user_data 关联到某个引擎，数据指针在使用期间必须一直有效。

高风险误区： 库不会擦写 H7 Flash。不要把工作区声明成未对齐 uint8_t 数组；使用公开 union。参数更新最好每个 DMA 半块只快照一次。

对应源码示例：`eurorack_algorithms/examples/plaits/plaits_example.c`

6.2 Braids 宏振荡器

用途	单音源合成器。一个振荡器里集合 48 类合成模型，可快速产生模拟波形、FM、人声、噪声和物理建模音色。
输入输出	无音频输入；输出单声道 int16；可选逐采样同步输入。
内存	上下文 17,152 B；无外部延迟线。
速率	固定 96 kHz。

最短正确调用流程

1. 静态声明 `aud_euro_braids_t`，调用 `aud_euro_braids_init`。
2. 用 `set_shape` 选择 0 到 47 的模型，用 `set_frequency_hz` 或 `set_pitch_q7` 定音高。
3. `timbre` 与 `color` 均按 0.0 到 1.0 更新；打击型模型需要 `aud_euro_braids_trigger`。
4. 在 96 kHz 音频回调中调用 `process`；48 kHz 系统要先做 2 倍采样率转换。

高风险误区： 不要把 96 kHz 输出简单每两个样点丢一个；高频模型会混叠。模型切换会改变参数含义，切换前应平滑增益。

对应源码示例：`eurorack_algorithms/examples/braids/braids_example.c`

6.3 Elements 物理建模音源

用途	用弓、气流、敲击激励模态共振器或波导弦，得到可演奏的真实质感声源。
输入输出	可选 blow/strike 浮点输入；main/aux 浮点输出；gate、MIDI 音高、调制和力度。
内存	上下文 113,664 B + 32,768 个 uint16 混响样点 65,536 B，共约 175 KiB。
速率	固定 32 kHz，内部安全块不超过 16 帧。

最短正确调用流程

1. 静态分配上下文和 reverb_buffer，调用 init。
2. 从 default_patch 开始，只逐步调 exciter、resonator、space 参数。
3. set_model 选择模态、单弦或弦乐合奏。gate 是持续电平；note 使用 MIDI 音符单位。
4. process_f32 可传 NULL 作为无外部激励，包装层自动静音填充并分成 16 帧块。

高风险误区： 上下文绝不能放 RTOS 任务栈。输出是 float，需要限幅并转换到编解码器格式；音频爆振时上游有 panic 保护但仍应限制参数。

对应源码示例：eurorack_algorithms/examples/elements/elements_example.c

6.4 Rings 共振器与波导弦

用途	把短脉冲、噪声或外部音频送入模态共振器、共鸣弦、波导弦、FM voice 与弦混响。
输入输出	可选 float 激励输入；main/aux float 输出；strum 是单次事件。
内存	上下文 110,592 B + 65,536 个 uint16 混响样点 131,072 B，共约 236 KiB。
速率	固定 48 kHz；内部块不超过 24 帧。

最短正确调用流程

1. 分配大上下文和 reverb_buffer，init 后选择模型和 1..4 复音数。
2. 从 default_patch 调结构、亮度、阻尼和激励位置。
3. internal_exciter=1 可在无音频输入时发声；strum 拉高一次后立即清零。
4. 包装层只在首个 24 帧子块消费 strum，避免大 DMA 缓冲重复拨弦。

高风险误区： 不要把 strum 当持续 gate。外部激励输入应限制在约 -1..1；输出在送 DAC 前限幅。

对应源码示例：[eurorack_algorithms/examples/rings/rings_example.c](#)

6.5 Clouds 颗粒与频谱处理器

用途	把输入录入循环缓冲区，再以多个短颗粒重放；也包含 WSOLA 拉伸、循环延迟和相位声码器。
输入输出	双声道 int16 输入和输出；trigger 是单次事件，gate/freeze 是持续电平。
内存	上下文 8,960 B + 大缓冲 118,784 B + 小缓冲 65,536 B，共约 188.8 KiB。
速率	固定 32 kHz。

最短正确调用流程

1. 用公开的 large/small buffer union 保证 8 字节对齐，并放入 AXI SRAM 或 SDRAM。
2. init 后选择 granular/stretch/looping delay/spectral 模式与单双声道质量。
3. 每个块先 set_parameters，再 process。trigger 在一次处理后由算法消费，应用侧也应清零。
4. 冻结时仍持续调用 process，否则内部读指针和效果尾音不会推进。

高风险误区： 48 kHz 输入不能直接喂给 32 kHz 核心。大缓冲不是 DMA 缓冲，通常不需要每块做 DCache 维护。

对应源码示例：[eurorack_algorithms/examples/clouds/clouds_example.c](#)

6.6 Warps 交叉调制、环调与声码器

用途	在两路音频之间做交叉淡化、波折叠、模拟/数字环调、XOR、比较器、声码器和频移。
输入输出	双声道 int16 输入被解释为 carrier/modulator；双声道输出为 main/aux。
内存	上下文 72,660 B。
速率	init 传实际采样率；每次 frames 必须是 3 的倍数，最大块由包装层自动切分。

最短正确调用流程

1. default_parameters 后 init，再设置两路 drive、algorithm、timbre。
2. carrier_shape=0 使用外部载波，非零时内部产生载波。
3. set_frequency_shifter 开启频移彩蛋路径。
4. 选择 48 或 96 帧 DMA 半缓冲可同时满足 3 的倍数约束。

高风险误区： 传非 3 倍数长度会返回 AUD_EURO_ERROR_RANGE 且不改输出，不能忽略返回值。drive 过大虽有饱和仍会明显改变频谱。

对应源码示例：eurorack_algorithms/examples/warps/warps_example.c

6.7 Peaks 包络、LFO 与鼓合成

用途	同一处理器可切换 ADSR/LFO、底鼓、军鼓、镲、FM 鼓、脉冲整形、弹跳球和迷你序列器。
输入输出	逐采样 gate_flags 输入；单声道 int16 输出。
内存	上下文 1,408 B。
速率	固定 48 kHz。

最短正确调用流程

1. init 传通道索引，set_full_control 选择四参数或简化控制。
2. set_function 选择 12 种功能；四个参数均为 0..65535。
3. process 的 gate_flags 每个样点使用 AUD_EURO_GATE_* 位。

高风险误区： 只传 HIGH 而不产生 RISING/FALLING 会让部分模式不触发。切功能时先做短淡出，防止状态突变产生爆音。

对应源码示例：[eurorack_algorithms/examples/peaks/peaks_example.c](#)

6.8 Tides 一代斜率发生器

用途	产生平滑、折叠、可同步的单周期/循环斜率，可作为 AD/AR 包络、LFO 或音频振荡器。
输入输出	逐采样控制标志；输出单极性、双极性和状态 flags。
内存	上下文 704 B。
速率	固定 48 kHz。

最短正确调用流程

1. init，选择 HIGH/MEDIUM/LOW 范围与 AD/LOOPING/AR 模式。
2. pitch_q7 用 MIDI 音符乘 128；slope/shape/smoothness 为有符号控制值。
3. process 持续运行；clock_sync 开启时给正确边沿 flags。

高风险误区： 高/中/低范围改变时间尺度。Tides 1 与 Tides 2 可以同时链接，构建系统已处理资源表重名。

对应源码示例：[eurorack_algorithms/examples/tides/tides_example.c](#)

6.9 Tides 2 四输出斜率发生器

用途	从一个斜率核心派生四路幅度、相位、频率或门关系，适合复合 LFO、包络和四相振荡。
输入输出	gate_flags 与可选 external_ramp；每帧四路 float 输出。
内存	上下文 248 B。
速率	任意采样率；init 传实际速率，process 直接传 Hz。

最短正确调用流程

- 1. init 传处理速率。
- 2. 选择 AD/LOOPING/AR、四种输出关系和 audio/control 范围。
- 3. frequency_hz 直接用 Hz；slope/shape/smoothness/shift 一般用 0..1。

高风险误区： external_ramp 为 NULL 表示内部斜坡；若使用外部锁定，数组长度必须与 frames 完全一致。

对应源码示例：[eurorack_algorithms/examples/tides2/tides2_example.c](#)

6.10 Stages 多段包络与序列器

用途	把 Ramp/Step/Hold/Alternating 段组合成 AD、ADSR、LFO、步进序列、延迟、滑音和 PLL 振荡器。
输入输出	逐采样 gate_flags; 输出 value、phase 和当前段号。
内存	上下文 3,096 B; 包装层最多 32 帧一块，限制上游 VLA 栈占用。
速率	固定 31.25 kHz。

最短正确调用流程

- 1. init 后构造 1..36 个 segment，每段给 type、loop、primary、secondary。
- 2. configure 选择普通触发或 sequencer 模式。
- 3. 每 32 us 运行，或在音频系统中独立定时运行并对输出保持/插值。

高风险误区： 不要直接按 48 kHz 调用并假装采样率相同；时常数会缩短。段参数含义随类型变化，先从两段 AD 示例开始。

对应源码示例: [eurorack_algorithms/examples/stages/stages_example.c](#)

6.11 Streams 动态与控制处理

用途	包含包络、vactrol、跟随器、压缩器、滤波控制和 Lorenz 混沌调制。
输入输出	int16 audio/excite 输入；输出 uint16 增益与滤波频率控制，而不是直接输出处理后音频。
内存	上下文 608 B。
速率	按应用处理速率逐样点运行；原模块控制路径与硬件 VCA/VCF 配合。

最短正确调用流程

1. init 传通道号，set_function 选择六种功能。
2. 设置两个 0..65535 参数；process 得到每帧 gain/filter_frequency。
3. 应用层用 gain 乘音频，或把 filter_frequency 映射到自己的滤波器截止频率。

高风险误区： 它不是一调用就得到压缩后 PCM 的黑盒效果器；原硬件由模拟 VCA/滤波器完成最后一步，H7 上要自行应用控制量。

对应源码示例：[eurorack_algorithms/examples/streams/streams_example.c](#)

6.12 Frames 关键帧与四路 LFO

用途	在最多 64 个时间点之间插值四路控制量，或生成带相位/频率扩散和耦合的四路 LFO。
输入输出	控制量输入输出，不直接处理音频。
内存	Keyframer 896 B；LFO 52 B。
速率	控制速率；LFO 也可按较高控制率运行。

最短正确调用流程

1. 初始化并 clear 关键帧表，为每路设置 easing 和 response。
2. 按 16 位 timestamp 添加四路 16 位值；evaluate 得到当前位置的四路电平。
3. LFO 路径单独初始化，设置 shape/spread/coupling 后周期性 render。

高风险误区： 关键帧插值不会替你把 0..65535 映射成电压或音频增益；映射和限幅属于应用层。

对应源码示例：[eurorack_algorithms/examples/frames/frames_example.c](#)

6.13 Marbles 相关随机门与随机电压

用途	产生带偏置、抖动、相关性、重复记忆和量化音阶的随机序列，适合测试激励与生成式控制。
输入输出	T 路输出门/斜坡，X/Y 路输出四路交错浮点随机电压。
内存	上下文 8,448 B。
速率	初始化时传采样率；可音频率或控制率运行。

最短正确调用流程

1. init 传实际处理速率与种子，并可持续 add_entropy 注入 ADC 噪声。
2. 从 default_t_parameters/default_group 开始配置。
3. 先 process_t 得到斜坡和两路门，再用同一斜坡调用 process_xy 得到四路电压。

高风险误区： gates 是每帧两元素交错数组，XY 是每帧四元素交错数组；长度算错会越界。随机源不能用于密码学。

对应源码示例：eurorack_algorithms/examples/marbles/marbles_example.c

6.14 Grids 鼓点地图与欧几里得节奏

用途	在二维鼓点拓扑图上插值底鼓/军鼓/镲节奏，也可生成三路欧几里得节奏。
输入输出	输入 24 PPQN 时钟脉冲；输出位图形式的门和重音。
内存	上游为全局单例，不能创建多个独立实例。
速率	每个外部时钟脉冲调用 tick；另按硬件定时调用 falling_edge/pulse_timer。

最短正确调用流程

- 1. default_settings 后配置 x/y、三路 density、randomness 和 swing。
- 2. init 传随机种子，再 configure。
- 3. 每个 24 PPQN 时钟调用 tick(1)，根据返回位图驱动 GPIO 或鼓音源触发。

高风险误区： 不要在多个任务并发调用。GPL-3.0-or-later；商业闭源固件若无法履行 GPL，应从工程中排除该对象。

对应源码示例：eurorack_algorithms/examples/grids/grids_example.c

6.15 Branches 伯努利门路由

用途	按概率把每次上升沿送到 A 或 B，可用于随机触发、自动测试序列和概率状态机。
输入输出	输入门电平和 A 概率；输出 A/B 门、是否产生新判决。
内存	几十字节；纯 C。
速率	控制速率，通常 1 kHz 或在 GPIO 边沿事件中调用。

最短正确调用流程

1. 初始化时给非零随机种子。
2. 每次采样都传入门电平，只有 0 到 1 的上升沿才生成新随机判决。
3. probability_a=0 永远选 B，=1 永远选 A；toggle/latch 决定翻转与保持行为。

高风险误区： 本模块为 GPL-3.0-or-later。概率参数已修正为直观 A 概率，不等同于原硬件旋钮的内部阈值方向。

对应源码示例：[eurorack_algorithms/examples/branches/branches_example.c](#)

6.16 Edges 风格芯片音振荡器

用途	提供三角波、NES 量化三角、移位寄存器噪声、正弦和降采样音色，并计算定时器周期。
输入输出	无输入；输出单声道 int16；gate 控制静音。
内存	小型 C 状态。
速率	可配置为编解码器实际采样率。

最短正确调用流程

- 1. init 传实际采样率。
- 2. set 传频率、波形和 gate，set_bitcrush 设置保持周期。
- 3. process 可处理任意帧数；timer_parameters 用于直接配置 H7 定时器 PWM。

高风险误区： 这是面向 H7 的等价功能实现，不是 AVR 定时器/GPIO 的逐周期仿真；GPL-3.0-or-later。

对应源码示例：[eurorack_algorithms/examples/edges/edges_example.c](#)

6.17 Yarns 自适应纯律与 MIDI 时钟

用途	公开 C API 提供根据当前和弦历史调整音高的纯律处理，以及与调用速率无关的 24 PPQN swing 时钟。
输入输出	note_on/off 输入 MIDI 音符；时钟 process 返回是否产生一个 MIDI Clock tick。
内存	上下文 192 B。
速率	控制速率可配置，例如精确 1 kHz。

最短正确调用流程

1. init 必须传实际、稳定的控制任务频率。
2. note_on 返回上游固定点音高偏移，note_off 维护当前和弦历史。
3. clock_start 传 BPM 和 swing 百分比，随后严格按初始化频率调用 clock_process。

高风险误区：完整 note stack、voice allocation、arpeggiator、sequencer 源码已保留在 vendor/yarns，但当前 C API 不模拟 Yarns 整机 CV/Gate 硬件。

对应源码示例：[eurorack_algorithms/examples/yarns/yarns_example.c](#)

6.18 FSK 解调与 CRC 数据包

用途	从比较器切片后的 0/1 采样中按脉宽区分 0、1、pause，再同步并验证 256 字节 CRC32 数据包。
输入输出	输入 uint8 逻辑采样；输出同步/解码/CRC 状态与只读 packet_data 指针。
内存	上下文 512 B。
速率	由发送端符号持续时间决定；pause/one/zero 参数以输入样点数表示。

最短正确调用流程

1. 先测出三类脉宽的典型样点数，必须 `pause > one > zero`。
2. `init` 后把比较器输出块持续送入 `process`。
3. 返回 `PACKET_OK` 时读取 256 字节数据；错误后调用 `sync` 重新搜索前导。

高风险误区： 它不是通用 BFSK 正交相关器，而是原音频引导协议的脉宽解码器；输入前需要比较器或施密特切片。

对应源码示例：[eurorack_algorithms/examples/fsk/fsk_example.c](#)

7. 27 个上游工程总表

工程	结果	可复用内容/结论
blades	仅硬件	模拟双多模滤波器；没有可移植 DSP 源码
blinds	仅硬件	模拟 VC 极化器；没有可移植 DSP 源码
braids	H7 C API	48-mode macro oscillator; band-limited analog oscillators; wavetable and wave-map synthesis; FM and feedback FM; vocal formant and VOSIM; physical-modeling voices; noise and particle synthesis; pitch quantizer and excitation envelope
branches	H7 C API	dual Bernoulli gate router; toggle and latch routing modes
clouds	H7 C API	granular synthesis; WSOLA time stretching; looping delay; phase vocoder and STFT ; pitch shifting; diffusion and reverb; mu-law audio storage; sample-rate conversion and correlation
ears	仅硬件	接触麦克风/包络硬件；没有可移植 DSP 源码
edges	H7 C API	triangle and NES triangle oscillators; pitched and NES noise; sample-rate reduction ; generic timer period and pulse-width calculation; MIDI note stack and voice allocation source
elements	H7 C API	modal resonator; waveguide string and string ensemble; bow, blow and strike excitors; tube model; multistage envelope; diffusion and reverb
frames	H7 C API	64-keyframe four-channel interpolator; step, linear, quartic, sine and bounce easing; four-channel phase/frequency-spread LFO
grids	H7 C API	topographic drum pattern interpolation; three-channel Euclidean rhythm generator; density, randomness, accents, swing and clock modes
kinks	仅硬件	整流/逻辑/采样保持硬件；没有可移植 DSP 源码
links	仅硬件	缓冲与混音硬件；没有可移植 DSP 源码
marbles	H7 C API	correlated random gate generator; Bernoulli, cluster, divider, drum and Markov models; deja-vu repeatable random sequences; quantized random voltage generator; clock recovery, ramp division and probability distributions
peaks	H7 C API	multistage envelope; free and tap-tempo LFO; bass, snare, hi-hat and FM drums; pulse shaper and randomizer; bouncing ball; mini sequencer and number station
plaits	H7 C API	24 macro-oscillator engines; virtual analog, waveshaping and phase distortion; two- and six-operator FM; grain, swarm, particle and noise synthesis; chords, string machine and chiptune; modal and string physical modeling; analog and synthetic drums; LPC, SAM and naive speech synthesis; wave terrain and user wavetable source
rings	H7 C API	modal resonator; sympathetic strings; waveguide string; FM voice; polyphonic strummer and plucker; chorus, ensemble and reverb
ripples	仅硬件	模拟多模滤波器；没有可移植 DSP 源码
shades	仅硬件	模拟衰减反相器；没有可移植 DSP 源码
shelves	仅硬件	模拟均衡器；没有可移植 DSP 源码
stages	H7 C API	36-stage ramp, step, hold and alternating segment generator; AD/ADSR envelopes and sequencer; tap LFO and PLL oscillator; sample-and-hold, delay and portamento ; audio-rate variable-shape oscillator
streams	H7 C API	envelope and vactrol models; envelope follower; compressor; filter controller; Lorenz chaotic modulation generator
tides	H7 C API	band-limited slope oscillator; AD, AR and looping envelope modes; wave shaping, smoothing and folding; clock synchronization and ratio prediction
tides2	H7 C API	four-output poly slope generator; amplitude, phase, frequency and gate output relationships; audio and control ranges; external-ramp locking, folding and filtering

工程	结果	可复用内容/结论
veils	仅硬件	模拟 VCA；没有可移植 DSP 源码
volts	仅硬件	电源工具；没有可移植 DSP 源码
warps	H7 C API	crossfade and cross-folding；analog and digital ring modulation；XOR and comparator modulation；vocoder and filter bank；quadrature transform and frequency shifter；oversampling and saturation
yarns	H7 C API	history-aware adaptive just intonation；sample-rate-independent 24-PPQN clock with swing；MIDI note stack, voice allocation, arpeggiator and sequencer source

“仅硬件”不是漏移植：这些工程的价值在模拟电路拓扑、器件与 PCB，而不是可在 MCU 上调用的软件。当前算法仓库没有复制硬件设计文件，因此也没有把 CC-BY-SA 硬件许可混入代码发布。

8. 对电赛信号题的实际作用

任务	优先模块	能做什么	是否建议比赛主路径
可编程信号源	Plaits, Braids, Edges, Tides2	正弦/复合波/FM/噪声/扫频和特殊测试激励	建议，先选简单确定模型
调制与频谱搬移	Warps	环调、乘法混频、比较/XOR、频移、声码器	适合验证或扩展
FSK 数据接收	FSK	比较器切片后的脉宽解调、同步和 CRC	协议匹配时建议
动态测量	Streams	包络、跟随、压缩增益控制	需自己应用增益/滤波
随机/伪随机激励	Marbles, Branches	概率门、相关随机序列	系统辨识/压力测试有用
谐振系统模拟	Rings, Elements	模态/弦/波导响应	展示型强，传统信号题通常非主线
时序与扫参	Frames, Stages, Yarns, Grids	关键帧、包络、时钟、节奏	自动测试与控制有用

电赛信号题通常优先考频率、幅度、相位、失真、滤波、调制解调和实时性。这个库最有价值的是快速提供成熟的信号源、非线性/调制算法和复杂测试激励；它不能替代 ADC 前端、抗混叠、精密时钟、校准、FFT/锁相环、测量不确定度和整机状态机。比赛主路径应选择最少、最确定的模块，不要为了“音色丰富”引入不必要的固定采样率和大内存负担。

9. 常见故障定位

现象	最可能原因	检查方法
一运行就 HardFault	大 context 放栈；工作区未对齐；DMA 访问 DTCM	看 map 文件、CFSR/BFAR、变量地址和链接段
周期性爆音	处理超过 DMA 截止时间；cache 未维护	DWT CYCCNT 测最大周期；GPIO 翻转看示波器
音高不准	采样率参数/codec 时钟不一致	测 LRCK，核对 init 传入速率与 PLL 配置
Clouds/Elements 音调异常	48 kHz 数据直接按 32 kHz 处理	关闭 SRC 对比；核对每秒实际调用帧数
Plaits 按住键反复触发	把 gate 当单次脉冲或块间强制清零	保持 trigger 高直到 NoteOff
Rings 一直重拨	strum 每块都为 1	只在边沿块置 1，调用后清零
Warps 输出不变	frames 不是 3 倍数，函数返回 RANGE	检查返回值，选 48/96 帧半缓冲
链接 Tides/Stages 报符号错	绕过 CMake 清单或未应用 Tides2 表重命名	使用提供的 CMake 或复制相同 source property
Keil static_assert context too small	使用旧头文件或不同 ABI	清理工程包含路径，确认当前常量与 ArmClang 6.24

10. 上板验收清单

1. 编译 map 中确认 context、delay/reverb、DMA 缓冲分别进入预期存储区，栈没有容纳大对象。
2. 用示波器测 codec LRCK/BCLK，确认实际采样率与算法一致。
3. 在音频回调入口/出口翻转 GPIO，并用 DWT CYCCNT 记录最大而非平均处理周期。
4. 分别播放静音、满幅正弦、脉冲和随机信号，检查 NaN、饱和、直流和左右声道布局。
5. 启用 DCache 后重复测试；若只在启用 cache 后损坏，先修 DMA cache/MPU，不要改 DSP 数学。
6. 对固定速率模块用频谱仪验证 SRC 镜像抑制和通带增益。
7. 对事件接口分别测试短按、长按、块边界上升沿和下降沿。
8. 连续运行至少 30 分钟，记录 underrun、HardFault、峰值周期和温度/电源状态。

只有完成此清单后，才能对某块具体 H7 硬件声称“运行验证通过”。当前仓库提供的是经过源码、双编译器和整库链接验证的可移植算法基线。

11. 许可证与发布

- 顶层现有 aud_*.c/h 为 DaisySP 衍生 MIT 代码，保留各文件通知。
- Mutable Instruments 的 STM32 DSP 算法、包装层与示例为 MIT，具体以文件头为准。
- Branches、Edges、Grids 及其兼容层/包装为 GPL-3.0-or-later。把它们链接进固件时，组合程序需要满足 GPL 传播义务。
- 不能满足 GPL 的工程应在 CMake 中排除这三个模块，而不是删除许可证头。
- Mutable Instruments 名称及模块名可能涉及商标；本仓库是独立移植，不是官方产品。

完整法律文本见仓库 LICENSE、LICENSES/MIT.txt 与 LICENSES/GPL-3.0-or-later.txt。

12. 文件索引

路径	用途
eurorack_algorithms/include	纯 C 公开 API；应用通常只 include aud_eurorack.h
eurorack_algorithms/port/src	C ABI 包装实现
eurorack_algorithms/port/include	去 HAL 兼容头、AVR shim、Plaits 用户数据桥
eurorack_algorithms/vendor	按上游项目分目录保存的 398 个算法/资源文件
eurorack_algorithms/examples	18 个可由 C 编译的项目示例
eurorack_algorithms/tests	C API 全模块链接冒烟测试
eurorack_algorithms/cmake	GNU Arm 与 Keil ArmClang H7 工具链
eurorack_algorithms/module_manifest.json	上游提交、项目算法、许可证和构建证据